Задание 1

Бизнес-процесс: покупка настольной игры через интернет-магазин с доставкой

Студент Вася хочет заказать настольную игру в подарок другу. Он заходит на сайт магазина, выбирает игру, оплачивает картой и указывает адрес доставки. Дальше магазин должен сам всё обработать: зарезервировать товар на складе, выбить чек, передать заказ в службу доставки и уведомить Васю.

В этом процессе участвуют несколько информационных систем:

Сайт интернет-магазина — витрина, где Вася выбирает игру.

CRM-система — хранит данные о клиентах, заказах и статусах.

Складская система (учёт товаров) — показывает, сколько коробок с игрой лежит на складе.

Платёжный шлюз (например, ЮKassa или Сбербанк) — списывает деньги с карты.

Бухгалтерская система (например, 1С) — формирует чек и учитывает выручку.

Служба доставки (СДЭК, Почта России или курьерская служба магазина) — везёт игру Васе.

Система уведомлений — отправляет Васе SMS или push-уведомления о статусе заказа.

Какие данные передаются между системами:

С сайта в CRM: номер заказа, название игры, артикул, количество (1 шт.), цена, ФИО клиента, телефон, email, адрес доставки, способ оплаты.

Из CRM в складскую систему: артикул и количество, чтобы зарезервировать товар. Склад отвечает: «Есть на складе» или «Нет в наличии».

Из CRM в платёжный шлюз: сумма к оплате, номер заказа, данные карты (в зашифрованном виде). Шлюз возвращает: «Оплата прошла» или «Ошибка».

Из CRM в бухгалтерию: данные для чека — название товара, цена, сумма, дата, номер заказа. Бухгалтерия отправляет чек в налоговую и клиенту.

Из CRM в службу доставки: адрес, ФИО, телефон, габариты и вес посылки, номер заказа. Служба доставки возвращает трек-номер для отслеживания.

Из CRM в систему уведомлений: телефон и email клиента, текст уведомления («Заказ принят», «Оплачен», «Передан курьеру», «Доставлен»).

Всё это должно работать автоматически, чтобы сотрудникам магазина не приходилось вручную переносить данные из одного окна в другое. Если интеграция настроена плохо, Вася может оплатить игру, а на складе её не зарезервируют — и тогда менеджеру придётся звонить и извиняться.

Задание 2

В лекции выделены три уровня интеграции:

- Интеграция на уровне данных.
- Интеграция на уровне приложений.
- Интеграция на уровне бизнес-процессов.

Рассмотрим магазин настольных игр и процесс покупки через интернет-магазин с доставкой.

1. На каком уровне интеграции процесс находится сейчас

Скорее всего, сейчас это уровень приложений (application level). Потому что системы уже общаются друг с другом через программные интерфейсы (API), а не просто через файлы или общую базу.

Например:

- Сайт через API передаёт заказ в CRM.

- CRM через API обращается к платёжному шлюзу, чтобы списать деньги.

- CRM через API передаёт данные в складскую систему, чтобы зарезервировать игру.

- CRM через API отправляет заказ в службу доставки и получает трек-номер.

Всё это происходит в реальном времени, без ручного переноса файлов. Значит, это не уровень данных, а именно уровень приложений.

Но внутри этого уровня интеграция построена по принципу «точка-точка» (point-to-point). То есть каждая система соединяется с каждой напрямую. Сайт знает, как разговаривать с CRM. CRM знает, как разговаривать с платежкой, складом, доставкой. Если завтра магазин захочет подключить новую систему (например, программу лояльности для постоянных клиентов), придётся писать новые интерфейсы для связи с каждой из уже существующих систем. Это неудобно и долго.

Если же в магазине всё делается вручную — менеджер скачивает заказы с сайта в Excel и загружает их в 1С, — то это был бы уровень данных. Но для современного магазина с сайтом и CRM более вероятен уровень приложений.

2. На какой уровень можно поднять

Процесс можно поднять на уровень бизнес-процессов (business process level). Это самый высокий уровень интеграции.

Что это значит на практике? Появляется центральная модель бизнес-процесса «Обработка заказа». Она описывает всю последовательность шагов:

1. Клиент оформляет заказ на сайте.

2. Система проверяет наличие игры на складе.

3. Если есть — принимает оплату.

4. После оплаты резервирует товар.

5. Передаёт заказ в службу доставки.

6. Отправляет клиенту уведомление.

7. Бухгалтерия формирует чек.

Эта модель не «зашита» в код каждой пары систем, а живёт отдельно — в интеграционной платформе (например, в BPM-системе или ESB). Все системы подключаются к этой платформе через стандартные интерфейсы. Платформа сама решает, какую систему в какой момент вызвать.

3. Как это выглядит в жизни

Сейчас: менеджер Вася сидит и вручную проверяет, оплатил ли клиент заказ, потом идёт на склад, потом звонит в доставку. Или системы дёргают друг друга напрямую, но если что-то сломалось, непонятно, где искать ошибку.

После перехода на уровень бизнес-процессов: Вася открывает одну панель управления и видит, что заказ №123 прошёл все шаги автоматически. Если на шаге «оплата» что-то пошло не так, система сама скажет: «Ошибка оплаты, свяжитесь с клиентом». И Вася не бегает по разным программам, а решает проблему в одном месте.

Задание 3

Для магазина настольных игр процесс покупки через сайт с доставкой может дать сбой на разных этапах. Вот две самые типичные ошибки.

Ошибка 1. Несогласованность справочников и форматов данных

Это когда одна и та же информация в разных системах записана по-разному. Например, на сайте игра называется «Каркассон», а в складской системе она лежит как «Carcassonne (база)». Или дата заказа на сайте указана в формате ДД.ММ.ГГГГ, а бухгалтерская программа ждёт ГГГГ-ММ-ДД. Из-за этого заказ не проходит автоматически: склад не может найти товар, бухгалтерия не выставляет чек. В итоге менеджеру приходится вручную исправлять данные, а это долго и чревато новыми ошибками.

Как предотвратить:

- Вести единый справочник товаров и клиентов для всех систем.

- Использовать стандартные форматы данных (например, для даты — ISO 8601).

- Настроить автоматическую проверку и преобразование форматов при передаче.

- Если систем много, лучше использовать интеграционную платформу (например, ESB), которая сама приводит данные к единому виду.

Ошибка 2. Потеря или дублирование заказа при сбое связи

Представим: студент Вася нажал «Оплатить», сайт отправил заказ в CRM, но из-за сбоя интернета ответ не пришёл. Сайт решает, что заказ не ушёл, и отправляет его повторно. В результате на складе появляются два одинаковых заказа. Или наоборот: заказ потерялся, деньги списались, а игра не зарезервирована. Клиент звонит, менеджер ищет концы.

Как предотвратить:

- Присваивать каждому заказу уникальный идентификатор и проверять его на стороне получателя. Если заказ с таким номером уже есть, второй раз его не создавать.

- Использовать очереди сообщений: если система не ответила, заказ не теряется, а ставится в очередь и повторяется позже.

- Настроить мониторинг и аудит всех операций обмена данными, чтобы сразу видеть, где произошёл сбой.

- Отказаться от ручного переноса данных — именно ручной ввод часто приводит к дублированию и потере информации.

Обе эти ошибки — прямое следствие того, что при интеграции не уделили внимания согласованности данных и надёжности передачи.

Задание 4

В магазине настольных игр подход «точка-точка» можно применить для связи интернет-магазина и складской системы.

Как это выглядит: когда студент Вася оформляет заказ на сайте, сайт напрямую обращается к складской системе через API. Складская система проверяет, есть ли игра в наличии, и если есть — резервирует её. Никакого центрального посредника (шины или интеграционной платформы) между ними нет. Это и есть подход «точка-точка»: приложения взаимодействуют напрямую друг с другом, и у каждой пары систем свой собственный интерфейс.

Почему для магазина игр это оправдано:

1. Мало систем. В этой конкретной связке участвуют всего две системы — сайт и склад. По формуле из лекции, при N = 2 количество интерфейсов равно 2 × (2 − 1) / 2 = 1. Один прямой интерфейс — это просто и понятно.

2. Простота и прозрачность. Не нужно покупать и настраивать дополнительное программное обеспечение. Всё работает напрямую, и если что-то сломалось, легко найти причину.

3. Магазин небольшой. У магазина пока нет десятка систем, которые нужно связывать между собой. Бизнес-процессы простые, и городить сложную интеграцию на уровне бизнес-процессов было бы избыточно и дорого.

4. Связь стабильная. Сайт и склад связаны постоянно, интерфейс меняется редко. Поэтому недостаток подхода «точка-точка» — квадратичный рост числа связей — здесь не проявляется.

Когда это перестанет быть оправданным: если магазин вырастет и захочет подключить ещё программы лояльности, службу доставки, бухгалтерию и т.д., то связывать каждую пару систем напрямую станет сложно.